

- Họ tên: Hồ Tuấn Huy
- MSSV: 20225856

Báo cáo kết quả bài tập phân tích từ vựng - Scanner

I. Kết quả thực hiện các ví dụ không có lỗi

- Kết quả thực hiện example1.kpl (không có lỗi):
 - Mã nguồn example1.kpl:

```
test >  example1.kpl  
  
1    Program Example1; (* Example 1 *)  
2    Begin  
3    End. (* Example 1 *)
```

- Kết quả:

```
● → GuiSV_Scanner ./scanner test/example1.kpl  
1-1:KW_PROGRAM  
1-9:TK_IDENT(Example1)  
1-17:SB_SEMICOLON  
2-1:KW_BEGIN  
3-1:KW_END  
3-4:SB_PERIOD
```

- Kết quả cho trước:

```
test >  result1.txt  
  
1    1-1:KW_PROGRAM  
2    1-9:TK_IDENT(Example1)  
3    1-17:SB_SEMICOLON  
4    2-1:KW_BEGIN  
5    3-1:KW_END  
6    3-4:SB_PERIOD
```

=> Chương trình đã phân tích thành công tất cả token trong file ví dụ 1 cho trước, kết quả cho hoàn toàn khớp với

kết quả được cho trước. Các từ khoá, định danh và ký hiệu được nhận diện đúng; comment được bỏ qua đúng cách.

- Kết quả thực hiện example2.kpl (không có lỗi):
 - Mã nguồn example2.kpl:

```
test >  example2.kpl

1  Program Example2; (* Factorial *)
2
3  Var n : Integer;
4
5  Function F(n : Integer) : Integer;
6      Begin
7          If n = 0 Then F := 1 Else F := N * F (N - 1);
8      End;
9
10 Begin
11     For n := 1 To 7 Do
12         Begin
13             Call WriteLn;
14             Call WriteI( F(i));
15         End;
16 End. (* Factorial *)
```

```
• → GuiSV_Scanner ./scanner test/example2.kpl
1-1:KW_PROGRAM
1-9:TK_IDENT(Example2)
1-17:SB_SEMICOLON
3-1:KW_VAR
3-5:TK_IDENT(n)
3-7:SB_COLON
3-9:KW_INTEGER
3-16:SB_SEMICOLON
5-1:KW_FUNCTION
5-10:TK_IDENT(F)
5-11:SB_LPAR
5-12:TK_IDENT(n)
5-14:SB_COLON
5-16:KW_INTEGER
5-23:SB_RPAR
5-25:SB_COLON
5-27:KW_INTEGER
5-34:SB_SEMICOLON
6-3:KW_BEGIN
7-5:KW_IF
7-8:TK_IDENT(n)
7-10:SB_EQ
7-12:TK_NUMBER(0)
7-14:KW_THEN
7-19:TK_IDENT(F)
7-21:SB_ASSIGN
7-24:TK_NUMBER(1)
7-26:KW_ELSE
7-31:TK_IDENT(F)
7-33:SB_ASSIGN
7-36:TK_IDENT(N)
7-38:SB_TIMES
7-40:TK_IDENT(F)
7-42:SB_LPAR
7-43:TK_IDENT(N)
7-45:SB_MINUS
7-47:TK_NUMBER(1)
7-48:SB_RPAR
7-49:SB_SEMICOLON
8-3:KW_END
8-6:SB_SEMICOLON
10-1:KW_BEGIN
```

```
10-1:KW_BEGIN
11-3:KW_FOR
11-7:TK_IDENT(n)
11-9:SB_ASSIGN
11-12:TK_NUMBER(1)
11-14:KW_TO
11-17:TK_NUMBER(7)
11-19:KW_DO
12-5:KW_BEGIN
13-7:KW_CALL
13-12:TK_IDENT(WriteLn)
13-19:SB_SEMICOLON
14-7:KW_CALL
14-12:TK_IDENT(WriteI)
14-18:SB_LPAR
14-20:TK_IDENT(F)
14-21:SB_LPAR
14-22:TK_IDENT(i)
14-23:SB_RPAR
14-24:SB_RPAR
14-25:SB_SEMICOLON
15-5:KW_END
15-8:SB_SEMICOLON
16-1:KW_END
16-4:SB_PERIOD
```

○ → GuiSV_Scanner 

- Kết quả:

=> Chương trình đã phân tích thành công tất cả token trong file ví dụ 2 cho trước, kết quả cho hoàn toàn khớp với kết quả được cho trước. Các từ khoá, định danh và ký hiệu được nhận diện đúng; comment được bỏ qua đúng cách.

- Kết quả thực hiện example3.kpl (không có lỗi):
 - Mã nguồn example3.kpl:

```
test > example3.kpl

1  PROGRAM  EXAMPLE3;  (* TOWER OF HANOI *)
2  VAR  I:INTEGER;
3      N:INTEGER;
4      P:INTEGER;
5      Q:INTEGER;
6      C:CHAR;
7
8  PROCEDURE  HANOI(N:INTEGER;  S:INTEGER;  Z:INTEGER);
9  BEGIN
10     IF  N != 0  THEN
11     BEGIN
12         CALL  HANOI(N-1,S,6-S-Z);
13         I:=I+1;
14         CALL  WRITELN;
15         CALL  WRITEI(I);
16         CALL  WRITEI(N);
17         CALL  WRITEI(S);
18         CALL  WRITEI(Z);
19         CALL  HANOI(N-1,6-S-Z,Z)
20     END
21 END;  (*END OF HANOI*)
22
23 BEGIN
24     FOR  N := 1  TO  4  DO
25     BEGIN
26         FOR  I:=1  TO  4  DO
27             CALL  WRITEC(' ');
28             CALL  READC(C);
29             CALL  WRITEC(C)
30         END;
31         P:=1;
32         Q:=2;
33         FOR  N:=2  TO  4  DO
34             BEGIN
35                 I:=0;
36                 CALL  HANOI(N,P,Q);
37                 CALL  WRITELN
38             END
39         END.  (* TOWER OF HANOI *)
```

- Kết quả:

```
● → GuiSV_Scanner ./scanner test/example3.kpl
1-1:KW_PROGRAM
1-10:TK_IDENT(EXAMPLE3)
1-18:SB_SEMICOLON
2-1:KW_VAR
2-6:TK_IDENT(I)
2-7:SB_COLON
2-8:KW_INTEGER
2-15:SB_SEMICOLON
3-6:TK_IDENT(N)
3-7:SB_COLON
3-8:KW_INTEGER
3-15:SB_SEMICOLON
4-6:TK_IDENT(P)
4-7:SB_COLON
4-8:KW_INTEGER
4-15:SB_SEMICOLON
5-6:TK_IDENT(Q)
5-7:SB_COLON
5-8:KW_INTEGER
5-15:SB_SEMICOLON
6-6:TK_IDENT(C)
6-7:SB_COLON
6-8:KW_CHAR
6-12:SB_SEMICOLON
8-1:KW_PROCEDURE
8-12:TK_IDENT(HANOI)
8-17:SB_LPAR
8-18:TK_IDENT(N)
8-19:SB_COLON
8-20:KW_INTEGER
8-27:SB_SEMICOLON
8-30:TK_IDENT(S)
8-31:SB_COLON
8-32:KW_INTEGER
8-39:SB_SEMICOLON
8-42:TK_IDENT(Z)
8-43:SB_COLON
8-44:KW_INTEGER
8-51:SB_RPAR
8-52:SB_SEMICOLON
9-1:KW_BEGIN
10-3:KW_IF
10-7:TK_IDENT(N)
10-9:SB_NEQ
10-12:TK_NUMBER(0)
10-15:KW_THEN
11-5:KW_BEGIN
12-7:KW_CALL
12-13:TK_IDENT(HANOI)
12-18:SB_LPAR
12-19:TK_IDENT(N)
12-20:SB_MINUS
12-20:SB_MINUS
12-21:TK_NUMBER(1)
12-22:SB_COMMA
12-23:TK_IDENT(S)
12-24:SB_COMMA
12-25:TK_NUMBER(6)
12-26:SB_MINUS
12-27:TK_IDENT(S)
12-28:SB_MINUS
12-29:TK_IDENT(Z)
12-30:SB_RPAR
12-31:SB_SEMICOLON
13-7:TK_IDENT(I)
13-8:SB_ASSIGN
13-10:TK_IDENT(I)
13-11:SB_PLUS
13-12:TK_NUMBER(1)
13-13:SB_SEMICOLON
14-7:KW_CALL
14-13:TK_IDENT(WRITELN)
14-20:SB_SEMICOLON
15-7:KW_CALL
15-13:TK_IDENT(WRITEI)
15-19:SB_LPAR
15-20:TK_IDENT(I)
15-21:SB_RPAR
15-22:SB_SEMICOLON
16-7:KW_CALL
16-13:TK_IDENT(WRITEI)
16-19:SB_LPAR
16-20:TK_IDENT(N)
16-21:SB_RPAR
16-22:SB_SEMICOLON
17-7:KW_CALL
17-13:TK_IDENT(WRITEI)
17-19:SB_LPAR
17-20:TK_IDENT(S)
17-21:SB_RPAR
17-22:SB_SEMICOLON
18-7:KW_CALL
18-13:TK_IDENT(WRITEI)
18-19:SB_LPAR
18-20:TK_IDENT(Z)
18-21:SB_RPAR
18-22:SB_SEMICOLON
19-7:KW_CALL
19-13:TK_IDENT(HANOI)
19-18:SB_LPAR
19-19:TK_IDENT(N)
19-20:SB_MINUS
19-21:TK_NUMBER(1)
19-22:SB_COMMA
19-23:TK_NUMBER(6)
```

19-23:TK_NUMBER(6)	29-7:KW_CALL
19-24:SB_MINUS	29-13:TK_IDENT(WRITEC)
19-25:TK_IDENT(S)	29-19:SB_LPAR
19-26:SB_MINUS	29-20:TK_IDENT(C)
19-27:TK_IDENT(Z)	29-21:SB_RPAR
19-28:SB_COMMA	30-5:KW_END
19-29:TK_IDENT(Z)	30-8:SB_SEMICOLON
19-30:SB_RPAR	31-3:TK_IDENT(P)
20-5:KW_END	31-4:SB_ASSIGN
21-1:KW_END	31-6:TK_NUMBER(1)
21-4:SB_SEMICOLON	31-7:SB_SEMICOLON
23-1:KW_BEGIN	32-3:TK_IDENT(Q)
24-3:KW_FOR	32-4:SB_ASSIGN
24-8:TK_IDENT(N)	32-6:TK_NUMBER(2)
24-10:SB_ASSIGN	32-7:SB_SEMICOLON
24-13:TK_NUMBER(1)	33-3:KW_FOR
24-16:KW_TO	33-8:TK_IDENT(N)
24-20:TK_NUMBER(4)	33-9:SB_ASSIGN
24-23:KW_DO	33-11:TK_NUMBER(2)
25-5:KW_BEGIN	33-14:KW_TO
26-7:KW_FOR	33-18:TK_NUMBER(4)
26-12:TK_IDENT(I)	33-21:KW_DO
26-13:SB_ASSIGN	34-5:KW_BEGIN
26-15:TK_NUMBER(1)	35-7:TK_IDENT(I)
26-18:KW_TO	35-8:SB_ASSIGN
26-22:TK_NUMBER(4)	35-10:TK_NUMBER(0)
26-25:KW_DO	35-11:SB_SEMICOLON
27-9:KW_CALL	36-7:KW_CALL
27-15:TK_IDENT(WRITEC)	36-13:TK_IDENT(HANOI)
27-21:SB_LPAR	36-18:SB_LPAR
27-22:TK_CHAR(' ')	36-19:TK_IDENT(N)
27-25:SB_RPAR	36-20:SB_COMMA
27-26:SB_SEMICOLON	36-21:TK_IDENT(P)
28-7:KW_CALL	36-22:SB_COMMA
28-13:TK_IDENT(READC)	36-23:TK_IDENT(Q)
28-18:SB_LPAR	36-24:SB_RPAR
28-19:TK_IDENT(C)	36-25:SB_SEMICOLON
28-20:SB_RPAR	37-7:KW_CALL
28-21:SB_SEMICOLON	37-13:TK_IDENT(WRITELN)
	38-5:KW_END
	39-1:KW_END
	39-4:SB_PERIOD

○ → **GuiSV_Scanner** ■

=> Chương trình đã phân tích thành công tất cả token trong file ví dụ 3 cho trước, kết quả cho hoàn toàn khớp với kết quả được cho trước. Các từ khoá, định danh và ký hiệu được nhận diện đúng; comment được bỏ qua đúng cách.

II. Kết quả thực hiện với các ví dụ có lỗi

- Ví dụ exampleA0.kpl (Lỗi ERR_INVALIDSYMBOL):

- Mã nguồn:

```
test > exampleA0.kpl
1  Program Example1; (* Example 1 *)
2  Begin
3  != & * := <= >'
4  '
5  End. (* Example 1 *)
```

- Để xử lý lỗi invalid symbol, cần sửa trong file scanner.c như sau:

```
return makeToken(SB_NEQ, lineNo, colNo-1);
case 22:
    token = makeToken(TK_NONE, lineNo, colNo-1);
    error(ERR_INVALIDSYMBOL, token->lineNo, token->colNo);
    return token;

case 43:
    token = makeToken(TK_NONE, lineNo, colNo);
    error(ERR_INVALIDSYMBOL, lineNo, colNo);
    readChar();
    return token;
default:
    token = makeToken(TK_NONE, lineNo, colNo);
    return token;
}
}
```

- Kết quả:

```

● → GuiSV_Scanner ./scanner test/exampleA0.kpl
1-1:KW_PROGRAM
1-9:TK_IDENT(Example1)
1-17:SB_SEMICOLON
2-1:KW_BEGIN
3-1:SB_NEQ
3-4:Invalid symbol!
3-4:TK_NONE
3-6:SB_TIMES
3-8:SB_ASSIGN
3-12:SB_LE
3-14:SB_GT
3-14:Invalid const char!
3-14:TK_NONE
4-0:Invalid const char!
4-0:TK_NONE
5-1:KW_END
5-4:SB_PERIOD
○ → GuiSV_Scanner █

```

- Ví dụ exampleA2.kpl (Lỗi ERR_INVALIDCHARCONSTANT):

- Mã nguồn:

```

test > exampleA2.kpl
1 Program DemoCharError;
2 Var
3   a, b : Char;
4 Begin
5   a := 'xy';      (* Lỗi: Hằng ký tự phải gồm đúng 1 ký tự *)
6   b := '
7   ';             (* Lỗi: Hằng ký tự không được xuống dòng *)
8 End.
9

```

- Để xử lý lỗi invalid const char, cần sửa trong file scanner.c như sau:

```
case 31:
    readChar();
    if (currentChar == EOF)
        state=34;
    else
        if(isprint(currentChar))
            state =32;
        else state =34;
    return getToken();
case 32:
    c= currentChar;
    ln = lineNo;
    cn = colNo - 1;
    readChar();
    if (charCodes[currentChar] == CHAR_SINGLEQUOTE)
        state=33;
    else
        state =34;
    return getToken();
case 33:
    token = makeToken(TK_CHAR, ln, cn);
    token->string[0] =c;
    token->string[1] ='\0';
    readChar();
    return token;
case 34:
    error(ERR_INVALIDCHARCONSTANT, lineNo, colNo-2);
    token = makeToken(TK_NONE, lineNo, colNo-2);
    return token;
```

- Kết quả:

```

● → GuiSV_Scanner ./scanner test/exampleA2.kpl
1-1:KW_PROGRAM
1-9:TK_IDENT(DemoCharError)
1-22:SB_SEMICOLON
2-1:KW_VAR
3-4:TK_IDENT(a)
3-5:SB_COMMA
3-7:TK_IDENT(b)
3-9:SB_COLON
3-11:KW_CHAR
3-15:SB_SEMICOLON
4-1:KW_BEGIN
5-4:TK_IDENT(a)
5-6:SB_ASSIGN
5-9:Invalid const char!
5-9:TK_NONE
5-11:TK_IDENT(y)
5-12:Invalid const char!
5-12:TK_NONE
6-4:TK_IDENT(b)
6-6:SB_ASSIGN
7--2:Invalid const char!
7--2:TK_NONE
7-1:Invalid const char!
7-1:TK_NONE
8-1:KW_END
9--1:SB_PERIOD
○ → GuiSV_Scanner □

```

- Ví dụ exampleA3.kpl (Lỗi ERR_ENDOFCOMMENT):

- Mã nguồn:

```

test >  exampleA3.kpl

1  Program Sample;
2  Begin
3      (* Đây là một comment
4      |   nhưng bị thiếu dấu đóng comment
5  End.

```

- Để xử lý lỗi end of comment expected, cần sửa trong file scanner.c như sau:

```
case 37:
    while (currentChar != EOF && charCodes[currentChar] != CHAR_TIMES) {
        readChar();
    }
    if (currentChar == EOF)
        state = 40;
    else
        state = 38;
    return getToken();
case 38:
    while (currentChar != EOF && charCodes[currentChar] == CHAR_TIMES) {
        readChar();
    }
    if (currentChar == EOF)
        state = 40;
    else if (charCodes[currentChar] == CHAR_RPAR)
        state = 39;
    else
        state = 37;
    return getToken();
case 39:
    readChar();
    state = 0;
    return getToken();
case 40:
    error(ERR_ENDOFCOMMENT, lineNo, colNo);
    token = makeToken(TK_NONE, lineNo, colNo);
    return token;
```

- Kết quả:

```
• → GuiSV_Scanner ./scanner test/exampleA3.kpl
1-1:KW_PROGRAM
1-9:TK_IDENT(Sample)
1-15:SB_SEMICOLON
2-1:KW_BEGIN
6-1:End of comment expected!
6-1:TK_NONE
○ → GuiSV_Scanner
```

- Ví dụ exampleA4.kpl (Lỗi ERR_IDENTTOOLONG):

- Mã nguồn:

```
test > exampleA4.kpl
1 Program DemoLongVar;
2
3 Var
4     thisVariableNameIsWayTooLongAndContains1234567890AndShouldCauseAnError : Integer;
5
6 Begin
7 End.
```

- Để xử lý lỗi identification too long, cần sửa trong file scanner.c như sau:
 - Ở case 3:
 - Reset cờ identTooLong = 0 khi bắt đầu.
 - Đọc toàn bộ định danh đến hết (kể cả khi vượt quá MAX_IDENT_LEN).
 - Nếu len >= MAX_IDENT_LEN thì set cờ identTooLong = 1 (không báo lỗi ngay).
 - Tiếp tục đọc đến khi gặp ký tự không phải chữ/số.

```

case 3:
{
    ln = lineNo;
    cn = colNo;
    int len = 0;
    identTooLong = 0;
    str[len++] = (char)currentChar;
    readChar();
    while (currentChar != EOF &&
           (charCodes[currentChar] == CHAR_LETTER ||
            charCodes[currentChar] == CHAR_DIGIT)) {
        if (len < MAX_IDENT_LEN) {
            str[len++] = (char)currentChar;
        } else {
            identTooLong = 1;
        }
        readChar();
    }
    if (len < MAX_IDENT_LEN) {
        str[len] = '\0';
    } else {
        str[MAX_IDENT_LEN - 1] = '\0';
    }
    state = 4;
    return getToken();
}

```

- Ở case 5:
 - Kiểm tra cờ identTooLong.
 - Nếu có cờ thì báo lỗi 1 lần, reset cờ và trả về TK_NONE.
 - Nếu không thì tạo token identifier bình thường

```

case 5:
    if (identTooLong) {
        error(ERR_IDENTTOOLONG, ln, cn);
        identTooLong = 0;
        return makeToken(TK_NONE, ln, cn);
    }
    token = makeToken(TK_IDENT, ln, cn);
    strcpy(token->string, str);
    return token;

```

- Reset cờ trong hàm scan() khi bắt đầu scan file mới.
- Kết quả:

```

● → GuiSV_Scanner ./scanner test/exampleA4.kpl
1-1:KW_PROGRAM
1-9:TK_IDENT(DemoLongVar)
1-20:SB_SEMICOLON
3-1:KW_VAR
4-5:Identification too long!
4-5:TK_NONE
4-76:SB_COLON
4-78:KW_INTEGER
4-85:SB_SEMICOLON
6-1:KW_BEGIN
7-1:KW_END
8--1:SB_PERIOD
○ → GuiSV_Scanner █

```